Text Summarization Report

A Production-Grade MLOps Capstone Project

Hugging Face Transformers · ZenML · MLflow · Optuna · Docker · GitHub Actions · Prometheus · Grafana

Submitted By

Mahi V Satani	2511042210020
Mouneeshwaran D	2511042210021
Saumya Patel	2511042210022

1 Objective / Problem Statement

1.1 Problem Statement

Manually reading and condensing large volumes of long-form news articles is slow and does not scale. The objective of this project is to design, build, and productionize an **abstractive text summarization service** that ingests a long article and returns a concise, coherent summary through a REST API, while being fully instrumented, tracked, tested, containerized, and monitored following MLOps best practices.

Concretely, the project delivers:

1. A **FastAPI** microservice (`/summarize`) that serves an abstractive summarization model (Hugging Face `seq2seq` model — BART/T5 family).
2. A **ZenML** pipeline that automates ingestion, validation, transformation, benchmarking/training, evaluation, and model registration/deployment.
3. **MLflow**-tracked experiments (model benchmarking + Optuna hyperparameter tuning) with the best model promoted to the **MLflow Model Registry**.
4. A **Dockerized** deployment (multi-service stack: API, MLflow, Prometheus, Grafana) runnable identically across local and cloud environments.
5. A **GitHub Actions** CI/CD pipeline that lints/tests every change and builds the Docker image before merge.
6. **Prometheus** metrics instrumented directly in the service code and a custom **Grafana** dashboard for real-time observability.

1.2 Business / Technical Relevance

- **Business relevance:** Automatic summarization reduces the time analysts, journalists, and knowledge workers spend triaging long documents, and can be embedded into content pipelines, dashboards, or chatbots as a reusable microservice.
- **Technical relevance:** The project is a compact but complete demonstration of the full MLOps lifecycle — reproducible pipelines, experiment tracking, automated tuning, model registry-driven deployment, containerization, CI/CD gating, and production monitoring — all applied to a real Hugging Face NLP model rather than a toy dataset.

1.3 Dataset Justification

The project uses the **CNN/DailyMail dataset** (`abisee/cnn_dailymail`, config `3.0.0`) via the Hugging Face `datasets` library.

- It is the **de-facto benchmark dataset** for abstractive summarization research, containing news articles (`article` column) paired with human-written bullet-point summaries (`highlights` column) — giving a ready-made reference for supervised ROUGE evaluation without any manual labeling effort.

- It is large enough to provide a **reliable validation split** for benchmarking and hyperparameter tuning, yet the code deliberately samples small, deterministic subsets (`BENCHMARK_SAMPLE_SIZE`, `TUNE_SAMPLE_SIZE`, `TEST_SAMPLE_SIZE`, all seeded with `seed=42`) to keep runtime inside the project's compute/time budget on CPU-only hardware.
- Using a standard, widely benchmarked dataset also makes the resulting ROUGE scores externally interpretable/comparable.

1.4 Success Metrics (defined upfront)

Metric	Definition	Target / Use
ROUGE-1 / ROUGE-2 / ROUGE-L (F1)	Overlap between generated summary and reference `highlights`	Primary model-quality metric; used to pick the best candidate model and best generation hyperparameters
Average inference latency (sec/request)	Wall-clock time per summarization call	Operational SLA metric, tracked per benchmark run and live via Prometheus
ROUGE-L regression threshold (0.12)	Minimum acceptable ROUGE-L in CI	Automated quality gate — a PR that degrades summarization quality below this bar fails CI
Service availability (`up` metric / `/health`)	Prometheus target health + FastAPI health endpoint	Deployment/production readiness signal
Request success rate	`summarization_service_requests_total{status="success"}` vs `{status="error"}`	Reliability of the deployed API

These metrics are logged to MLflow during benchmarking/tuning, asserted in the CI regression test, and re-observed live in production via the Prometheus/Grafana stack — so the same metric definitions apply from experimentation through to production monitoring.

1.5 Scope vs. the 15-Hour Budget

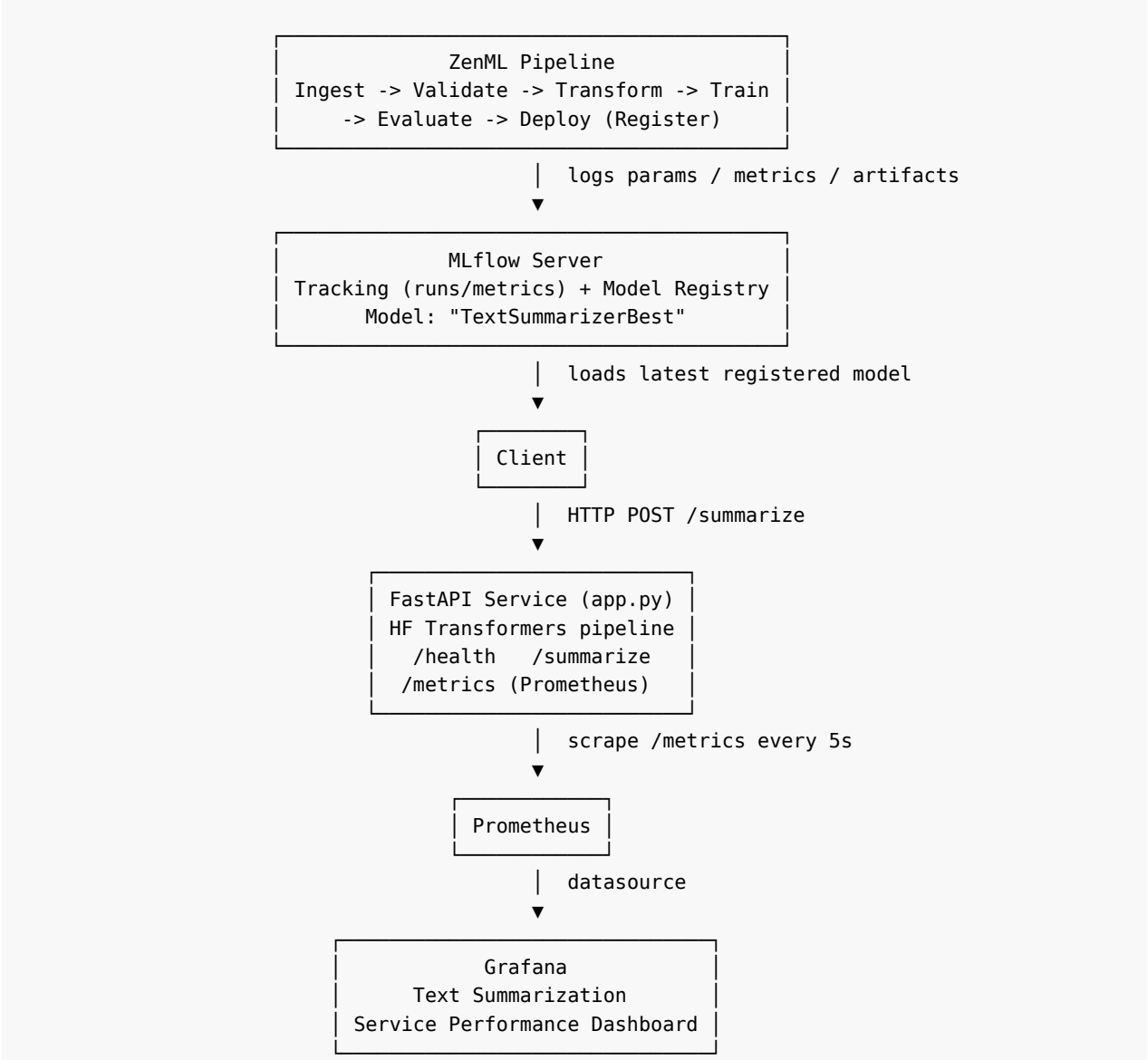
The scope is deliberately narrow and CPU-friendly so that it is realistically achievable within a 15-hour build budget:

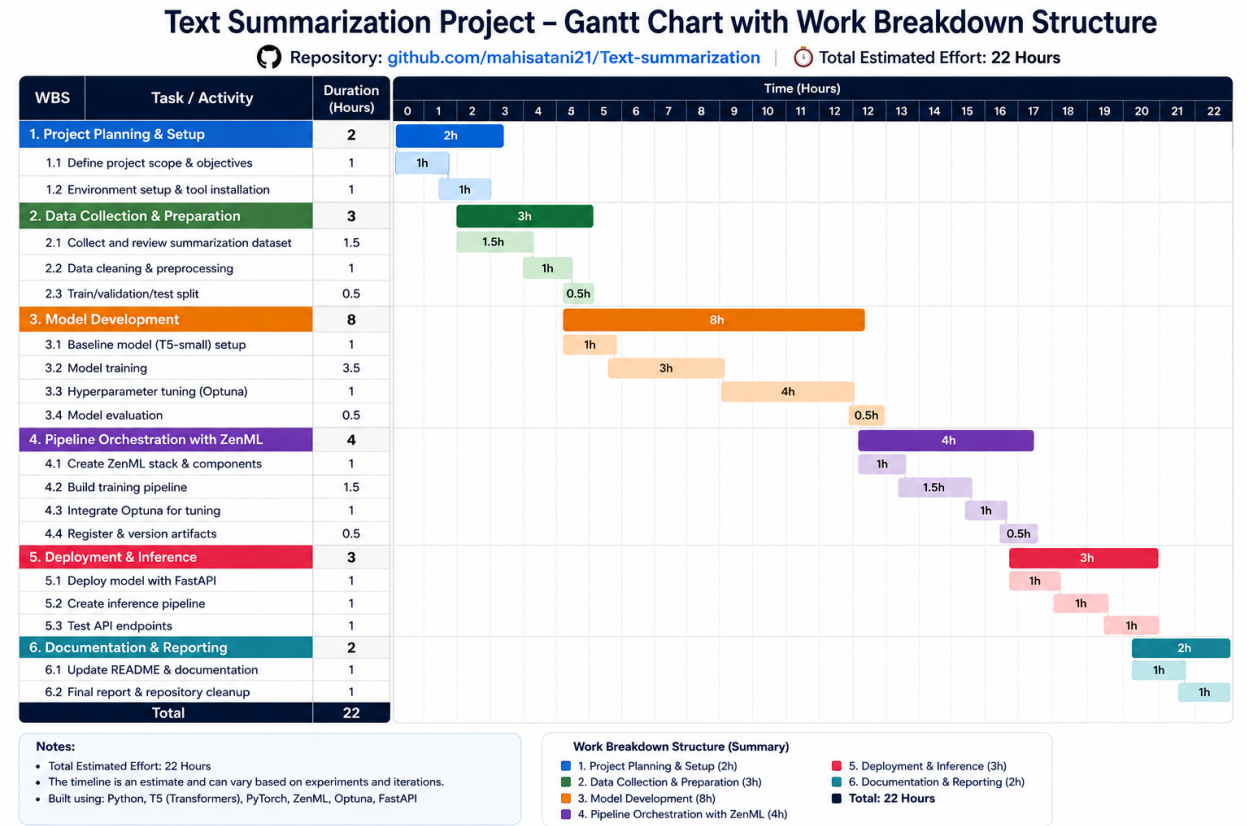
- **3 candidate models only** (`sshleifer/distilbart-cnn-6-6`, `t5-small`, `google/flan-t5-small`) rather than a large model zoo.
- **Small, seeded data samples** (5–10 articles) for benchmarking/tuning instead of the full validation split (~13k articles), which would be computationally infeasible on CPU within the time budget.

- **Optuna tuning restricted to 2 generation hyperparameters** (`num_beams`, `length_penalty`) over the already-selected best base model, rather than re-tuning model weights — this keeps the search space small and each trial cheap (no re-training), which is the correct trade-off for an LLM/seq2seq project per the evaluation notes.
- **A single registered "best" model** (`TextSummarizerBest`) is promoted and served, rather than a multi-model ensemble.

---

2 System Architecture





All services above are orchestrated together via `docker-compose`, and the whole stack is gated on every push/PR by the GitHub Actions CI pipeline (lint/test + Docker build).

---

3 Project Structure

```
text-summarization-service/
├── .github/
│   └── workflows/
│       └── ci.yml                # GitHub Actions CI pipeline
├── config/
│   ├── prometheus.yml            # Prometheus scrape configuration
│   └── grafana/
│       ├── datasources.yml       # Grafana → Prometheus datasource provisioning
│       ├── dashboards.yml        # Grafana dashboard provider config
│       └── dashboard_summarization.json  # Grafana dashboard (as code)
├── docker/
│   ├── Dockerfile                # Production image for the FastAPI service
│   └── docker-compose.yml        # Full stack: app + mlflow + prometheus + grafana
├── src/
│   ├── app.py                    # FastAPI service (inference + /health + /metrics)
│   ├── config.py                 # Centralized configuration / env vars
│   ├── metrics.py                # Prometheus metric definitions
```

```
│   ├── train.py                  # Candidate model benchmarking (MLflow logged)
│   ├── tune.py                   # Optuna hyperparameter tuning + model registration
│   ├── utils.py                  # Dataset loading + ROUGE scoring utilities
│   └── zenml_pipeline.py         # Full ZenML pipeline (ingest→...→deploy)
├── tests/
│   └── test_rouge_regression.py  # ROUGE regression test used in CI
├── mlflow_export.py               # Exports MLflow runs to CSV for offline analysis
├── mlflow_runs.csv                # Exported experiment run history
├── requirements.txt                # Pinned Python dependencies
├── .gitignore                      # Excludes venvs, caches, mlruns/, secrets, etc.
├── Run_project.txt                 # Step-by-step run instructions
└── README.md                       # Project documentation (setup, MLflow/Prometheus/Grafana
↪  guides)
```

This structure keeps orchestration (`src/zenml_pipeline.py`), the serving layer (`src/app.py`), configuration (`config/`), deployment (`docker/`), automated tests (`tests/`), and CI (`.github/workflows/`) cleanly separated, with dependencies pinned in `requirements.txt` and no secrets, virtual environments, `mlruns/`, or model binaries committed to the repository (enforced via `.gitignore`).

```
# .gitignore
venv/
.venv/
Env/
env/
__pycache__/
*.pyc
*.pyo
*.pyd
.pytest_cache/
.coverage
htmlcov/
mlruns/
mlartifacts/
mlflow.db
mlflow.db-journal
local_mlflow_model/
.vscode/
.idea/
.agents/
*.log
```

---

4 Configuration (`src/config.py`)

All tunable parameters — MLflow URIs, dataset name/version, sample sizes, candidate models, and default generation settings — are centralized in a single configuration module and overridable via environment variables, so the same code runs unchanged locally, in CI, and in Docker.

```python
# src/config.py
import os

# MLflow Configurations
MLFLOW_TRACKING_URI = os.getenv("MLFLOW_TRACKING_URI", "http://localhost:5000")
MLFLOW_EXPERIMENT_NAME = os.getenv("MLFLOW_EXPERIMENT_NAME", "Text_Summarization_System")
MODEL_REGISTRY_NAME = os.getenv("MODEL_REGISTRY_NAME", "TextSummarizerBest")

# Dataset Configurations
DATASET_NAME = "abisee/cnn_dailymail"
DATASET_VERSION = "3.0.0"
# Use a small subset for benchmarking and hyperparameter tuning in local/CI runs
BENCHMARK_SAMPLE_SIZE = int(os.getenv("BENCHMARK_SAMPLE_SIZE", "10"))
TUNE_SAMPLE_SIZE = int(os.getenv("TUNE_SAMPLE_SIZE", "10"))
TEST_SAMPLE_SIZE = int(os.getenv("TEST_SAMPLE_SIZE", "5"))

# Candidate Models for Benchmarking
CANDIDATE_MODELS = [
    "sshleifer/distilbart-cnn-6-6",
    "t5-small",
    "google/flan-t5-small"
]

# Default best model to fall back on if registry is unavailable or for local default
↪  initialization
DEFAULT_MODEL_NAME = os.getenv("DEFAULT_MODEL_NAME", "sshleifer/distilbart-cnn-6-6")

# Generation Parameters Configuration
DEFAULT_MIN_LENGTH = 30
DEFAULT_MAX_LENGTH = 142
```

Key design points:

- `MLFLOW_TRACKING_URI`, sample sizes, and the default model are all env-overridable — this is what lets `docker-compose.yml` and `ci.yml` set smaller `TUNE_SAMPLE_SIZE`/`TEST_SAMPLE_SIZE` for fast, deterministic pipeline runs without touching code.
- `CANDIDATE_MODELS` explicitly scopes benchmarking to three lightweight, CPU-friendly seq2seq models, matching the 15-hour compute budget.

---

5 Shared Utilities (`src/utils.py`)

Dataset loading and ROUGE scoring are implemented once and reused by `train.py`, `tune.py`, `zenml_pipeline.py`, and `tests/test_rouge_regression.py` — avoiding duplicated evaluation logic across the pipeline.

```python
# src/utils.py
from datasets import load_dataset
from rouge_score import rouge_scorer
import pandas as pd
from typing import List, Dict
import logging
from src.config import DATASET_NAME, DATASET_VERSION

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def load_data_sample(split: str = "validation", sample_size: int = 10) -> pd.DataFrame:
    """
    Loads a small sample from the CNN/DailyMail dataset.
    """
    logger.info(f"Loading {split} split of {DATASET_NAME} (config: {DATASET_VERSION})...")
    try:
        dataset = load_dataset(DATASET_NAME, DATASET_VERSION, split=split)
    except Exception as e:
        logger.warning(f"Failed to load {DATASET_NAME} with version {DATASET_VERSION}, trying
↪   config name directly: {e}")
        dataset = load_dataset("cnn_dailymail", "3.0.0", split=split)

    # Shuffling and selecting a deterministic sample
    shuffled_dataset = dataset.shuffle(seed=42)
    sample = shuffled_dataset.select(range(min(sample_size, len(dataset))))

    df = pd.DataFrame(sample)
    logger.info(f"Loaded {len(df)} samples from {split} split.")
    return df

def calculate_rouge_scores(predictions: List[str], references: List[str]) -> Dict[str, float]:
    """
    Calculates ROUGE-1, ROUGE-2, and ROUGE-L F1 scores for predictions and references.
    """
    scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)

    total_scores = {'rouge1': 0.0, 'rouge2': 0.0, 'rougeL': 0.0}
    n = len(predictions)
    if n == 0:
        return total_scores

    for pred, ref in zip(predictions, references):
        scores = scorer.score(ref, pred)
        total_scores['rouge1'] += scores['rouge1'].fmeasure
        total_scores['rouge2'] += scores['rouge2'].fmeasure
        total_scores['rougeL'] += scores['rougeL'].fmeasure

    avg_scores = {k: v / n for k, v in total_scores.items()}
    return avg_scores
```

---

6 ML Pipeline — ZenML Orchestration (`src/zenml_pipeline.py`)

The project implements a **full ZenML pipeline** with six distinct, reusable, independently-testable `@step`-decorated functions, wired together in a single `@pipeline` function: **ingest → validate → transform → train → evaluate → deploy**. Each step has typed inputs/outputs, which ZenML uses to cache and version pipeline runs automatically.

6.1 Step 1 — Data Ingestion

```python
# --- STEP 1: Ingest ---
@step
def data_ingest_step() -> pd.DataFrame:
    """
    Ingests validation data from CNN/DailyMail dataset.
    """
    logger.info("Starting Data Ingestion Step...")
    # Load a small sample size of 5 for demonstration purposes
    df = load_data_sample(split="validation", sample_size=5)
    return df
```

Pulls a deterministic, seeded sample of the CNN/DailyMail validation split via the shared `load_data_sample` utility.

6.2 Step 2 — Data Validation (with pipeline-halting checks)

```python
# --- STEP 2: Validate ---
@step
def data_validate_step(df: pd.DataFrame) -> pd.DataFrame:
    """
    Validates that the dataset schema is correct, no null values exist,
    and columns are present. Halts pipeline on failure.
    """
    logger.info("Starting Data Validation Step...")

    # 1. Check if DataFrame is empty
    if df.empty:
        raise ValueError("Validation Failed: The dataset is empty!")

    # 2. Check schema (column existence)
    required_columns = ["article", "highlights"]
    for col in required_columns:
        if col not in df.columns:
            raise ValueError(f"Validation Failed: Missing required column '{col}'!")

    # 3. Check for null values in required columns
    null_counts = df[required_columns].isnull().sum()
```

```
    for col, count in null_counts.items():
        if count > 0:
            raise ValueError(f"Validation Failed: Column '{col}' contains {count} null values!")

    logger.info("Data Validation Passed: Schema is correct and no null values found.")
    return df
```

This step **actively** checks (a) that the dataframe is non-empty, (b) that the expected schema (`article`, `highlights` columns) is present, and (c) that there are no null values in required columns. Any failure raises a `ValueError`, which **halts pipeline execution** before a bad dataset can reach training — satisfying the requirement that validation can actively stop the pipeline, not just log a warning.

6.3 Step 3 — Data Transformation

```
# --- STEP 3: Transform ---
@step
def data_transform_step(df: pd.DataFrame) -> pd.DataFrame:
    """
    Transforms/prepares the text by truncating long inputs to avoid model constraints.
    """
    logger.info("Starting Data Transformation Step...")
    df_transformed = df.copy()

    # Basic text cleaning: strip trailing/leading spaces and truncate to 4000 characters
    df_transformed["article"] = df_transformed["article"].apply(lambda x: str(x).strip()[:4000])
    df_transformed["highlights"] = df_transformed["highlights"].apply(lambda x: str(x).strip())

    logger.info("Data Transformation complete.")
    return df_transformed
```

Cleans whitespace and truncates overly long articles (character-level cap) so inputs stay within the token limits of the candidate seq2seq models.

6.4 Step 4 — Model Training / Benchmarking

```
# --- STEP 4: Train ---
@step
def model_train_step(df: pd.DataFrame) -> str:
    """
    Simulates training/benchmarking of candidate models on validation samples.
    Logs metrics and parameters to MLflow. Returns the best model name.
    """
    logger.info("Starting Model Training/Benchmarking Step...")

    mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
```

```
    mlflow.set_experiment(MLFLOW_EXPERIMENT_NAME)

    articles = df["article"].tolist()
    references = df["highlights"].tolist()

    device = 0 if torch.cuda.is_available() else -1
    best_model_name = DEFAULT_MODEL_NAME
    best_rouge_l = -1.0

    # Benchmark the candidate models
    for model_name in CANDIDATE_MODELS[:2]:  # Limit to first two for speed
        logger.info(f"Benchmarking model: {model_name}")

        with mlflow.start_run(run_name=f"zenml_benchmark_{model_name.replace('/', '_')}"):
            mlflow.log_param("model_name", model_name)
            mlflow.log_param("dataset_size", len(articles))

            try:
                tokenizer = AutoTokenizer.from_pretrained(model_name)
                model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
                summarizer = pipeline(
                    "summarization",
                    model=model,
                    tokenizer=tokenizer,
                    device=device
                )

                start_time = time.time()
                predictions = []
                for article in articles:
                    summary = summarizer(
                        article,
                        max_length=142,
                        min_length=30,
                        do_sample=False
                    )[0]['summary_text']
                    predictions.append(summary)

                elapsed_time = time.time() - start_time
                avg_latency = elapsed_time / len(articles)

                scores = calculate_rouge_scores(predictions, references)
                rouge_l = scores["rougeL"]

                mlflow.log_metric("avg_latency_sec", avg_latency)
                mlflow.log_metric("rouge1", scores["rouge1"])
                mlflow.log_metric("rouge2", scores["rouge2"])
                mlflow.log_metric("rougeL", rouge_l)

                logger.info(f"Model: {model_name} | ROUGE-L: {rouge_l:.4f}")

                if rouge_l > best_rouge_l:
                    best_rouge_l = rouge_l
                    best_model_name = model_name
```

```
            except Exception as e:
                logger.error(f"Error benchmarking {model_name}: {e}")
                mlflow.log_param("status", "failed")

    logger.info(f"Training/benchmarking complete. Best model: {best_model_name}")
    return best_model_name
```

Every candidate model is benchmarked in its **own MLflow run**, logging `model_name`, `dataset_size`, `avg_latency_sec`, `rouge1`, `rouge2`, and `rougeL` as parameters/metrics. The model with the highest ROUGE-L is selected as `best_model_name`.

6.5 Step 5 — Model Evaluation

```
# --- STEP 5: Evaluate ---
@step
def model_evaluate_step(df: pd.DataFrame, best_model_name: str) -> dict:
    """
    Performs a final evaluation on the best model, outputting a dictionary of scores.
    """
    logger.info(f"Starting Final Model Evaluation Step for model: {best_model_name}")

    articles = df["article"].tolist()
    references = df["highlights"].tolist()
    device = 0 if torch.cuda.is_available() else -1

    tokenizer = AutoTokenizer.from_pretrained(best_model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(best_model_name)
    summarizer = pipeline(
        "summarization",
        model=model,
        tokenizer=tokenizer,
        device=device
    )

    predictions = []
    for article in articles:
        summary = summarizer(
            article,
            max_length=142,
            min_length=30,
            do_sample=False
        )[0]['summary_text']
        predictions.append(summary)

    scores = calculate_rouge_scores(predictions, references)
    logger.info(f"Final Evaluation Scores: {scores}")
    return scores
```

Re-evaluates the selected best model on the transformed data to produce a final

ROUGE score dictionary, independent from the benchmarking loop, as a sanity check before deployment.

6.6 Step 6 — Model Deployment / Registration

```
# --- STEP 6: Deploy / Register ---
@step
def model_deploy_step(best_model_name: str) -> None:
    """
    Registers the best model in the MLflow Model Registry.
    """
    logger.info(f"Starting Model Registry/Deployment Step for model: {best_model_name}")

    mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
    mlflow.set_experiment(MLFLOW_EXPERIMENT_NAME)

    tokenizer = AutoTokenizer.from_pretrained(best_model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(best_model_name)

    with mlflow.start_run(run_name="zenml_deploy") as run:
        components = {
            "model": model,
            "tokenizer": tokenizer,
        }
        # Log and register best model in model registry
        mlflow.transformers.log_model(
            transformers_model=components,
            artifact_path="best_model",
            task="summarization",
            registered_model_name=MODEL_REGISTRY_NAME,
            model_config={
                "num_beams": 4,
                "length_penalty": 2.0,
                "max_length": 142,
                "min_length": 30,
                "do_sample": False
            }
        )
    logger.info(f"Successfully registered model '{best_model_name}' under registry name
↪  '{MODEL_REGISTRY_NAME}'")
```

Logs and registers the winning model under the fixed registry name `TextSummarizerBest`, which the FastAPI service later loads at startup.

6.7 Pipeline Definition

```
# --- ZENML PIPELINE DEFINITION ---
@zenml_pipeline
```

```
def text_summarization_pipeline():
    """
    Full ZenML orchestration pipeline:
    ingest -> validate -> transform -> train -> evaluate -> deploy
    """
    data = data_ingest_step()
    validated_data = data_validate_step(df=data)
    transformed_data = data_transform_step(df=validated_data)
    best_model = model_train_step(df=transformed_data)
    evaluation_metrics = model_evaluate_step(df=transformed_data, best_model_name=best_model)
    model_deploy_step(best_model_name=best_model)


if __name__ == "__main__":
    # Ensure ZenML uses local tracking
    os.environ["ZENML_DEBUG"] = "true"
    os.environ["PYTHONIOENCODING"] = "utf-8"

    # Run the pipeline
    text_summarization_pipeline()
```

Because every step is typed and side-effect-free with respect to ZenML's artifact store, **ZenML automatically versions and caches each pipeline run** — re-running the pipeline with unchanged upstream artifacts skips re-computation of unaffected steps.

---

7 Experiment Tracking — MLflow + Model Benchmarking (`src/train.py`)

Independent of the ZenML pipeline, `src/train.py` performs a standalone benchmarking sweep across **all three** candidate models (the ZenML step limits itself to 2 for speed) and logs each as its own MLflow run.

```
# src/train.py
import time
import mlflow
import torch
from transformers import pipeline, AutoModelForSeq2SeqLM, AutoTokenizer
from src.config import (
    MLFLOW_TRACKING_URI,
    MLFLOW_EXPERIMENT_NAME,
    CANDIDATE_MODELS,
    BENCHMARK_SAMPLE_SIZE
)
from src.utils import load_data_sample, calculate_rouge_scores
import logging

logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)

def benchmark_models():
    # Set up MLflow
    mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
    mlflow.set_experiment(MLFLOW_EXPERIMENT_NAME)

    # Load dataset sample
    df = load_data_sample(split="validation", sample_size=BENCHMARK_SAMPLE_SIZE)
    articles = df['article'].tolist()
    references = df['highlights'].tolist()

    device = 0 if torch.cuda.is_available() else -1
    logger.info(f"Using device: {'GPU' if device == 0 else 'CPU'}")

    best_model_name = None
    best_rouge_l = -1.0

    for model_name in CANDIDATE_MODELS:
        logger.info(f"Benchmarking model: {model_name}")

        # Start MLflow run
        with mlflow.start_run(run_name=f"benchmark_{model_name.replace('/', '_')}"):
            # Log params
            mlflow.log_param("model_name", model_name)
            mlflow.log_param("dataset_size", len(articles))
            mlflow.log_param("device", "cuda" if device == 0 else "cpu")

            try:
                # Load tokenizer and model
                tokenizer = AutoTokenizer.from_pretrained(model_name)
                model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

                # Setup Pipeline
                summarizer = pipeline(
                    "summarization",
                    model=model,
                    tokenizer=tokenizer,
                    device=device
                )

                # Perform inference and time it
                start_time = time.time()
                predictions = []
                for article in articles:
                    # Truncate article to avoid overflow (most models handle max 1024 or 512
                    ↪  tokens)
                    truncated_article = article[:4000] # simple character truncation to avoid
↪  huge inputs
                    try:
                        summary = summarizer(
                            truncated_article,
                            max_length=142,
                            min_length=30,
```

```
                        do_sample=False
                    )[0]['summary_text']
                except Exception as e:
                    logger.error(f"Inference failed for article: {e}")
                    summary = ""
                predictions.append(summary)

            elapsed_time = time.time() - start_time
            avg_latency = elapsed_time / len(articles)

            # Compute ROUGE scores
            scores = calculate_rouge_scores(predictions, references)

            # Log metrics
            mlflow.log_metric("avg_latency_sec", avg_latency)
            mlflow.log_metric("rouge1", scores['rouge1'])
            mlflow.log_metric("rouge2", scores['rouge2'])
            mlflow.log_metric("rougeL", scores['rougeL'])

            # Log Model artifact using mlflow.transformers
            components = {
                "model": model,
                "tokenizer": tokenizer,
            }
            mlflow.transformers.log_model(
                transformers_model=components,
                artifact_path="model",
                task="summarization"
            )

            logger.info(f"Model: {model_name} | ROUGE-L: {scores['rougeL']:.4f} | Avg
↪  Latency: {avg_latency:.2f}s")

            if scores['rougeL'] > best_rouge_l:
                best_rouge_l = scores['rougeL']
                best_model_name = model_name

        except Exception as e:
            logger.error(f"Failed to benchmark model {model_name}: {e}")
            mlflow.log_param("status", "failed")
            mlflow.log_param("error_message", str(e))

    logger.info(f"Benchmarking complete. Best model is {best_model_name} with ROUGE-L of
↪  {best_rouge_l:.4f}")
    return best_model_name

if __name__ == "__main__":
    benchmark_models()
```

Every run logs **parameters** (`model_name`, `dataset_size`, `device`), **metrics** (`avg_latency_sec`, `rouge1`, `rouge2`, `rougeL`), and an **artifact** (the packaged HF model+tokenizer via `mlflow.transformers.log_model`) — satisfying full experiment logging (params, metrics, artifacts) for every run.

8 Hyperparameter Tuning — Optuna + MLflow Model Registry (`src/tune.py`)

```python
# src/tune.py
import os
import mlflow
import optuna
import torch
import logging
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline
from src.config import (
    MLFLOW_TRACKING_URI,
    MLFLOW_EXPERIMENT_NAME,
    MODEL_REGISTRY_NAME,
    TUNE_SAMPLE_SIZE,
    DEFAULT_MODEL_NAME
)
from src.utils import load_data_sample, calculate_rouge_scores

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def objective(trial, model_name, articles, references, device):
    # Suggest hyperparameters
    num_beams = trial.suggest_int("num_beams", 1, 6)
    length_penalty = trial.suggest_float("length_penalty", 0.5, 2.5)

    # Start nested MLflow run for each trial
    with mlflow.start_run(run_name=f"trial_{trial.number}", nested=True):
        mlflow.log_params({
            "num_beams": num_beams,
            "length_penalty": length_penalty,
            "trial_number": trial.number
        })

        try:
            predictions = []
            for article in articles:
                truncated_article = article[:4000]
                summary = trial.user_attrs["pipeline"](
                    truncated_article,
                    max_length=142,
                    min_length=30,
                    num_beams=num_beams,
                    length_penalty=length_penalty,
                    do_sample=False
                )[0]['summary_text']
                predictions.append(summary)
```

```
            scores = calculate_rouge_scores(predictions, references)
            score = scores['rougeL']

            # Log metrics
            mlflow.log_metric("rouge1", scores['rouge1'])
            mlflow.log_metric("rouge2", scores['rouge2'])
            mlflow.log_metric("rougeL", score)

            return score
        except Exception as e:
            logger.error(f"Trial {trial.number} failed: {e}")
            mlflow.log_param("status", "failed")
            return 0.0

def tune_hyperparameters(model_name: str = None, n_trials: int = None):
    if n_trials is None:
        n_trials = int(os.getenv("N_TRIALS", "3"))
    if not model_name:
        model_name = os.getenv("BEST_MODEL_NAME", DEFAULT_MODEL_NAME)

    mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
    mlflow.set_experiment(MLFLOW_EXPERIMENT_NAME)

    logger.info(f"Tuning generation parameters for model: {model_name}")

    df = load_data_sample(split="validation", sample_size=TUNE_SAMPLE_SIZE)
    articles = df['article'].tolist()
    references = df['highlights'].tolist()

    device = 0 if torch.cuda.is_available() else -1

    # Load tokenizer and model once to avoid reload overhead
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
    summarizer = pipeline(
        "summarization",
        model=model,
        tokenizer=tokenizer,
        device=device
    )

    # Parent MLflow Run for the study
    with mlflow.start_run(run_name=f"optuna_tuning_{model_name.replace('/', '_')}") as
    ↪  parent_run:
        mlflow.log_params({
            "model_name": model_name,
            "tuning_samples": len(articles),
            "n_trials": n_trials
        })

        study = optuna.create_study(direction="maximize")

        # Inject the pipeline into study/trials attributes so the objective can access it
        study.enqueue_trial({"num_beams": 4, "length_penalty": 2.0}) # standard default settings
```

```
        def lambda_obj(trial):
            trial.set_user_attr("pipeline", summarizer)
            return objective(trial, model_name, articles, references, device)

        study.optimize(lambda_obj, n_trials=n_trials)

        logger.info(f"Best trial parameters: {study.best_params}")
        logger.info(f"Best ROUGE-L: {study.best_value:.4f}")

        # Log best parameters and metrics in the parent run
        mlflow.log_params({f"best_{k}": v for k, v in study.best_params.items()})
        mlflow.log_metric("best_rougeL", study.best_value)

        # Log and Register the Best Model in registry
        components = {
            "model": model,
            "tokenizer": tokenizer,
        }

        # We save model with the best parameters in metadata / config
        model_info = mlflow.transformers.log_model(
            transformers_model=components,
            artifact_path="best_model",
            task="summarization",
            registered_model_name=MODEL_REGISTRY_NAME,
            model_config={
                "num_beams": study.best_params.get("num_beams", 4),
                "length_penalty": study.best_params.get("length_penalty", 1.0),
                "max_length": 142,
                "min_length": 30,
                "do_sample": False
            }
        )

        logger.info(f"Best model registered under: {MODEL_REGISTRY_NAME}")
        return study.best_params

if __name__ == "__main__":
    tune_hyperparameters()
```

Tuning design (and why the trial budget is justified):

- **Sampler:** Optuna's default `TPESampler` (Tree-structured Parzen Estimator) via `optuna.create_study(direction="maximize")`, maximizing `rougeL`.
- **Search space:** `num_beams` (int, 1–6) and `length_penalty` (float, 0.5–2.5) — the two generation-time hyperparameters with the largest effect on abstractive summary quality/latency trade-off.
- **Warm start:** `study.enqueue_trial({"num_beams": 4, "length_penalty": 2.0})` seeds the search with HF's conventional defaults before exploring further.
- **Nested MLflow runs:** each Optuna trial is logged as a **nested run** under the parent tuning run, so the full trial history (params + `rouge1`/`rouge2`/`rougeL` per trial)

is inspectable in the MLflow UI (Parallel Coordinates / Scatter plots, as documented in the project README).

- **Trial budget (`n_trials`):** defaults to 3 locally (configurable via `N_TRIALS`) because this is a **compute-heavy Hugging Face seq2seq inference workload on CPU** — each trial re-runs generation over the full tuning sample. Per the project's own evaluation guidance, an LLM/HF fine-tuning-style task is expected to run **far fewer trials than a tabular model**, and tuning effort should be judged proportionally to the 15-hour time budget rather than by raw trial count. Because the model itself is *not* re-trained per trial (only generation hyperparameters change), each trial is still meaningfully informative despite the small budget.
- **Registry promotion:** the trial with the best `rougeL` has its parameters embedded in the registered model's `model_config`, and the model+tokenizer are logged and **registered** under `MODEL_REGISTRY_NAME = "TextSummarizerBest"` via `mlflow.transformers.log_model(..., registered_model_name=...)` — this is the "best run promoted via MLflow Model Registry" requirement.

---

9 Exporting Experiment History (`mlflow_export.py`)

```python
# mlflow_export.py
import os
import pandas as pd
import mlflow
from src.config import MLFLOW_TRACKING_URI, MLFLOW_EXPERIMENT_NAME

def export_runs_to_csv(output_path: str = "mlflow_runs.csv"):
    # Set tracking URI
    mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)

    # Get experiment
    experiment = mlflow.get_experiment_by_name(MLFLOW_EXPERIMENT_NAME)
    if not experiment:
        print(f"Experiment '{MLFLOW_EXPERIMENT_NAME}' not found.")
        return

    print(f"Fetching runs for experiment: '{MLFLOW_EXPERIMENT_NAME}' (ID:
    ↪  {experiment.experiment_id})...")

    # Search runs
    runs_df = mlflow.search_runs(experiment_ids=[experiment.experiment_id])

    if runs_df.empty:
        print("No runs found in this experiment.")
        return

    # Export to CSV
    runs_df.to_csv(output_path, index=False)
```

```
    print(f"Successfully exported {len(runs_df)} runs to {output_path}")

if __name__ == "__main__":
    export_runs_to_csv()
```

This utility queries the MLflow tracking server via `mlflow.search_runs()` and exports the complete run history (params, metrics, tags, timestamps) to `mlflow_runs.csv` for offline analysis in Pandas/Excel — used as a durable, versionable snapshot of experimentation, independent of the live MLflow server.

---

10 Serving Layer — FastAPI Inference Service (`src/app.py`)

The production service loads the **latest registered model** from the MLflow Model Registry at startup, falls back gracefully to a local Hugging Face default if the registry is unreachable, and exposes health, inference, and Prometheus metrics endpoints.

```
# src/app.py
import time
import os
import logging
from contextlib import asynccontextmanager
from typing import Optional

from fastapi import FastAPI, HTTPException, Response
from fastapi.responses import PlainTextResponse, RedirectResponse
from pydantic import BaseModel, Field
import mlflow
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline
from prometheus_client import generate_latest, CONTENT_TYPE_LATEST, REGISTRY

from src.config import (
    MLFLOW_TRACKING_URI,
    MODEL_REGISTRY_NAME,
    DEFAULT_MODEL_NAME,
    DEFAULT_MIN_LENGTH,
    DEFAULT_MAX_LENGTH
)
from src.metrics import (
    LATENCY_HISTOGRAM,
    INPUT_TOKEN_HISTOGRAM,
    OUTPUT_TOKEN_HISTOGRAM,
    REQUEST_COUNTER
)

# Setup logging
logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)

# Global variables for model, tokenizer, and configuration
summarizer_pipeline = None
model_tokenizer = None
active_model_name = None
generation_config = {}

class SummarizeRequest(BaseModel):
    text: str = Field(..., description="The text content to be summarized")
    num_beams: Optional[int] = Field(None, ge=1, le=10, description="Beam width for beam
↪   search")
    length_penalty: Optional[float] = Field(None, ge=0.0, le=5.0, description="Length penalty
↪   for generation")

class SummarizeResponse(BaseModel):
    summary: str
    latency_sec: float
    input_tokens: int
    output_tokens: int
    model_used: str

@asynccontextmanager
async def lifespan(app: FastAPI):
    global summarizer_pipeline, model_tokenizer, active_model_name, generation_config

    device = 0 if torch.cuda.is_available() else -1
    logger.info(f"App starting up. Using device: {'GPU' if device == 0 else 'CPU'}")

    # Try loading from MLflow Model Registry
    loaded_from_registry = False
    # Reachability check with a short timeout to prevent client blocking when MLflow is offline
    mlflow_reachable = False
    if MLFLOW_TRACKING_URI:
        try:
            import urllib.request
            urllib.request.urlopen(MLFLOW_TRACKING_URI, timeout=1.5)
            mlflow_reachable = True
        except Exception:
            pass

    try:
        if not mlflow_reachable:
            raise ConnectionError("MLflow tracking server is not reachable")
        mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
       logger.info(f"Attempting to load best model '{MODEL_REGISTRY_NAME}' from MLflow registry
↪   at {MLFLOW_TRACKING_URI}...")
        # Get the latest version in the registry
        client = mlflow.tracking.MlflowClient()
        latest_versions = client.get_latest_versions(MODEL_REGISTRY_NAME, stages=["None",
↪   "Production", "Staging"])

        if latest_versions:
            latest_version = latest_versions[0].version
```

```
            model_uri = f"models:/{MODEL_REGISTRY_NAME}/{latest_version}"
            logger.info(f"Found model version {latest_version}. Loading from {model_uri}...")

            # Ensure destination directory exists
            os.makedirs("./local_mlflow_model", exist_ok=True)
            # Load registered model using mlflow.transformers
            model_info = mlflow.transformers.load_model(model_uri,
↪  dst_path="./local_mlflow_model")

            # Extract tokenizer, model and configuration
            if hasattr(model_info, 'tokenizer') and hasattr(model_info, 'model'):
                model_tokenizer = model_info.tokenizer
                summarizer_pipeline = pipeline(
                    "summarization",
                    model=model_info.model,
                    tokenizer=model_tokenizer,
                    device=device
                )
            else:
                model_tokenizer = model_info.get("tokenizer")
                summarizer_pipeline = pipeline(
                    "summarization",
                    model=model_info.get("model"),
                    tokenizer=model_tokenizer,
                    device=device
                )

            if hasattr(model_info, 'model_config'):
                generation_config = model_info.model_config

            active_model_name = f"Registry:{MODEL_REGISTRY_NAME}:v{latest_version}"
            loaded_from_registry = True
            logger.info("Successfully loaded model from MLflow registry.")
        else:
            logger.warning(f"No versions found in MLflow registry for model:
↪  {MODEL_REGISTRY_NAME}")

    except Exception as e:
        logger.warning(f"Could not load model from MLflow Registry: {e}. Falling back to default
↪  model: {DEFAULT_MODEL_NAME}")

    # Fallback to local default pre-trained HF model if registry failed/empty
    if not loaded_from_registry:
        try:
            logger.info(f"Loading fallback default model '{DEFAULT_MODEL_NAME}' from Hugging
↪  Face...")
            model_tokenizer = AutoTokenizer.from_pretrained(DEFAULT_MODEL_NAME)
            model = AutoModelForSeq2SeqLM.from_pretrained(DEFAULT_MODEL_NAME)
            summarizer_pipeline = pipeline(
                "summarization",
                model=model,
                tokenizer=model_tokenizer,
                device=device
            )
```

```
            active_model_name = f"HF:{DEFAULT_MODEL_NAME}"
            generation_config = {
                "num_beams": 4,
                "length_penalty": 2.0,
                "max_length": DEFAULT_MAX_LENGTH,
                "min_length": DEFAULT_MIN_LENGTH,
                "do_sample": False
            }
            logger.info("Successfully loaded fallback model.")
        except Exception as fallback_err:
            logger.critical(f"Failed to load fallback model: {fallback_err}")
            raise RuntimeError(f"No model available to serve: {fallback_err}")

    yield
    # Shutdown logic
    logger.info("App shutting down.")

app = FastAPI(
    title="Text Summarization API",
    description="Production-grade HF Text Summarization service with MLflow tracking and
↪   Prometheus metrics.",
    version="1.0.0",
    lifespan=lifespan
)

@app.get("/")
async def root():
    """
    Redirects the root URL to the API documentation (/docs).
    """
    return RedirectResponse(url="/docs")

@app.get("/health", status_code=200)
async def health_check():
    """
    Health check endpoint for container orchestrators.
    """
    if summarizer_pipeline is None:
        raise HTTPException(status_code=503, detail="Model is not initialized.")
    return {
        "status": "healthy",
        "model": active_model_name,
        "device": str(summarizer_pipeline.device)
    }

@app.post("/summarize", response_model=SummarizeResponse)
async def summarize(request: SummarizeRequest):
    """
    Generates a summary for the provided input text.
    """
    if not request.text.strip():
        REQUEST_COUNTER.labels(status="error").inc()
        raise HTTPException(status_code=400, detail="Input text cannot be empty.")
```

```
    start_time = time.time()

    try:
        # Determine number of tokens in input
        input_tokens = len(model_tokenizer.encode(request.text, truncation=True))
        INPUT_TOKEN_HISTOGRAM.observe(input_tokens)

        # Override parameters if provided in request, else use model defaults
        num_beams = request.num_beams if request.num_beams is not None else
↪   generation_config.get("num_beams", 4)
        length_penalty = request.length_penalty if request.length_penalty is not None else
↪   generation_config.get("length_penalty", 2.0)

        # Run summarization
        # Char limit truncation as safety
        truncated_text = request.text[:8000]

        summary_result = summarizer_pipeline(
            truncated_text,
            max_length=DEFAULT_MAX_LENGTH,
            min_length=DEFAULT_MIN_LENGTH,
            num_beams=num_beams,
            length_penalty=length_penalty,
            do_sample=False
        )

        summary_text = summary_result[0]['summary_text']

        # Determine output token length
        output_tokens = len(model_tokenizer.encode(summary_text))
        OUTPUT_TOKEN_HISTOGRAM.observe(output_tokens)

        latency = time.time() - start_time
        LATENCY_HISTOGRAM.observe(latency)

        # Increment request counter
        REQUEST_COUNTER.labels(status="success").inc()

        return SummarizeResponse(
            summary=summary_text,
            latency_sec=latency,
            input_tokens=input_tokens,
            output_tokens=output_tokens,
            model_used=active_model_name
        )

    except Exception as e:
        logger.error(f"Error during summarization: {e}")
        REQUEST_COUNTER.labels(status="error").inc()
        raise HTTPException(status_code=500, detail=f"Inference error: {str(e)}")

@app.get("/metrics")
async def get_metrics():
    """
```

```
    Exposes metrics for Prometheus scraping.
    """
    return Response(content=generate_latest(REGISTRY), media_type=CONTENT_TYPE_LATEST)
```

Notable production-readiness details:

- **Registry-first, HF-fallback loading:** the service attempts to load `TextSummarizerBest` from MLflow with a 1.5-second reachability probe before falling back to a hardcoded default HF model — the API never fails to start just because MLflow is temporarily unavailable.
- **`/health`** returns `503` until the model is initialized, and reports the active model name/device — used by both Docker `HEALTHCHECK` and orchestration/load balancer probes.
- **Per-request Prometheus instrumentation:** input/output token histograms and latency are observed on every `/summarize` call, and a labeled `status="success"/"error"` counter tracks reliability.
- **`/metrics`** exposes the Prometheus text-exposition format directly from the FastAPI process (not from a sidecar), satisfying the requirement that instrumentation live in the service code itself.
- Auto-generated interactive documentation is available at **`/docs`** (Swagger UI), shown in the screenshots below.

10.1 Live API Documentation & Inference Evidence

The screenshot above (`localhost:8002/docs`) confirms all four endpoints defined in `app.py` are live and correctly documented via FastAPI's auto-generated OpenAPI schema (`Text Summarization API`, version `1.0.0`).

This confirms end-to-end functionality: a real article about the Curiosity rover was summarized in **5.6 seconds**, consuming **73 input tokens** and producing **52 output tokens**, using the model registered in Step 6 of the ZenML pipeline (`model_used: "Registry:TextSummarizerBest:v1"`).

---

11 Prometheus Instrumentation (`src/metrics.py`)

Metrics are defined once as module-level Prometheus client objects and imported directly into `app.py`, so instrumentation lives in the application code rather than being bolted on at the infrastructure layer.

```
# src/metrics.py
from prometheus_client import Histogram, Counter

# Namespace or prefix for all metrics
```

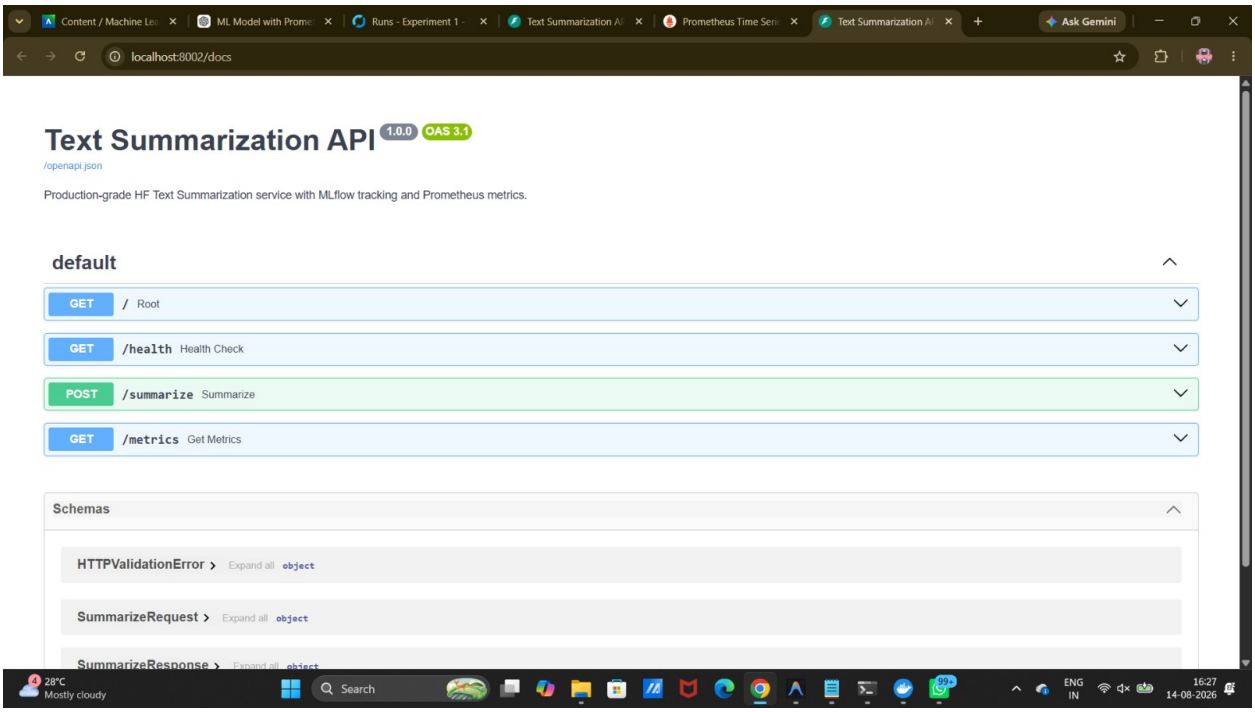


Figure 1: FastAPI interactive documentation (Swagger UI) listing the four exposed endpoints: GET /, GET /health, POST /summarize, GET /metrics.

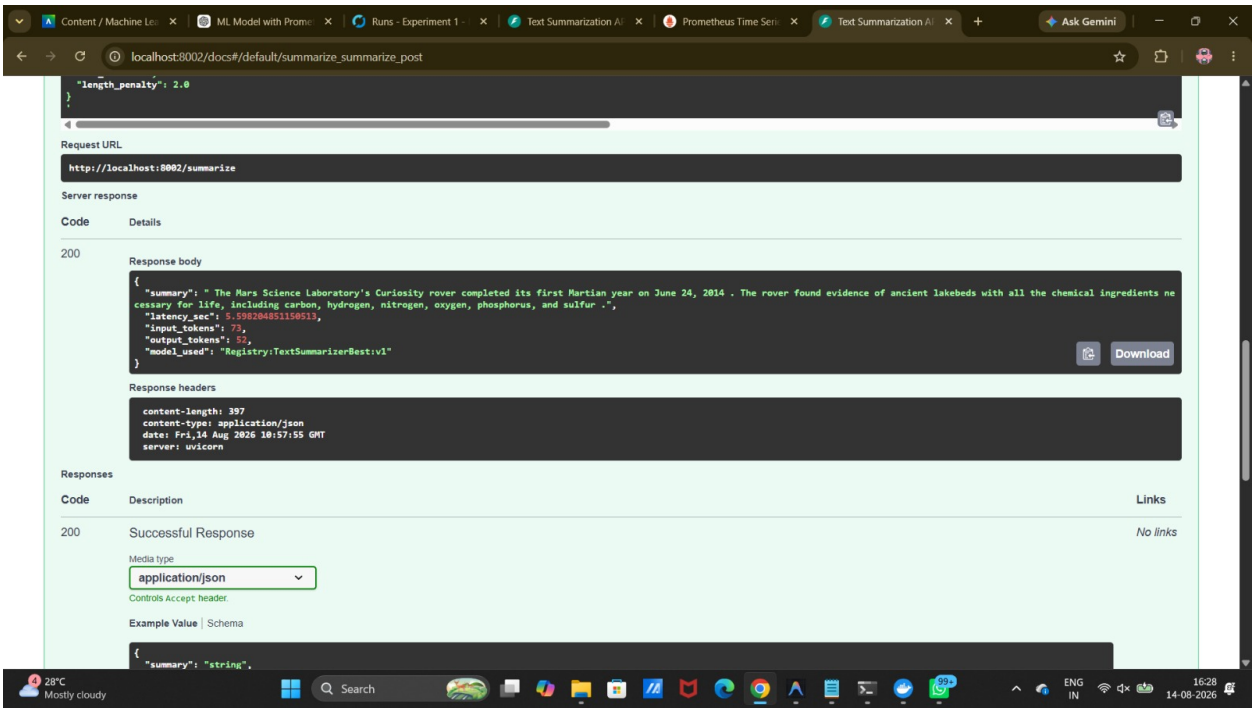


Figure 2: A live `POST /summarize` call against `http://localhost:8002/summarize`, returning a 200 response with the generated summary, latency (`5.598 s`), input/output token counts, and the active model identifier `Registry:TextSummarizerBest:v1` — confirming the model was served from the MLflow Model Registry, not the fallback path.

```
NAMESPACE = "summarization_service"

# Request Latency Metric
LATENCY_HISTOGRAM = Histogram(
    "request_latency_seconds",
    "Time taken to process summarization request in seconds",
    namespace=NAMESPACE,
    buckets=[0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0, 30.0, 60.0]
)

# Input token length distribution
INPUT_TOKEN_HISTOGRAM = Histogram(
    "input_token_length",
    "Number of tokens in the input text",
    namespace=NAMESPACE,
    buckets=[10, 50, 100, 250, 500, 750, 1000, 1500, 2000]
)

# Output token length distribution
OUTPUT_TOKEN_HISTOGRAM = Histogram(
    "output_token_length",
    "Number of tokens in the generated summary",
    namespace=NAMESPACE,
    buckets=[5, 10, 20, 30, 50, 70, 100, 120, 150, 200]
)

# Request counter for overall traffic stats
REQUEST_COUNTER = Counter(
    "requests_total",
    "Total number of summarization requests processed",
    namespace=NAMESPACE,
    labelnames=["status"]
)
```

This yields four metrics under the `summarization_service_` namespace:

Metric	Type	Purpose
`summarization_service_request_latency_seconds`	Histogram	Request-count metric (latency) — powers p95/p99 latency panels
`summarization_service_input_token_length`	Histogram	Domain-specific metric — distribution of input article length
`summarization_service_output_token_length`	Histogram	Domain-specific metric — distribution of generated summary length
`summarization_service_requests_total{status}`	Counter	Request-count / error-rate metric, labeled by `success`/`error`

This satisfies the requirement of tracking, at minimum, **request count, latency,**

error rate, and a domain-specific metric (here, two: input/output token-length distributions).

11.1 Prometheus Scrape Configuration (`config/prometheus.yml`)

```yaml
global:
  scrape_interval: 5s
  evaluation_interval: 5s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "summarization_service"
    metrics_path: "/metrics"
    static_configs:
      - targets: ["app:8000"]
```

Prometheus scrapes the FastAPI service's `/metrics` endpoint every 5 seconds using the Docker Compose service name `app:8000` for service discovery.

11.2 Prometheus Query Evidence

These confirm the target health check `up{job="summarization_service"}` would report `1` (healthy) and that raw counter data is queryable directly from Prometheus before it ever reaches Grafana.

---

12 Grafana Dashboard — Built From Scratch

12.1 Datasource Provisioning (`config/grafana/datasources.yml`)

```yaml
apiVersion: 1

datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: true
```

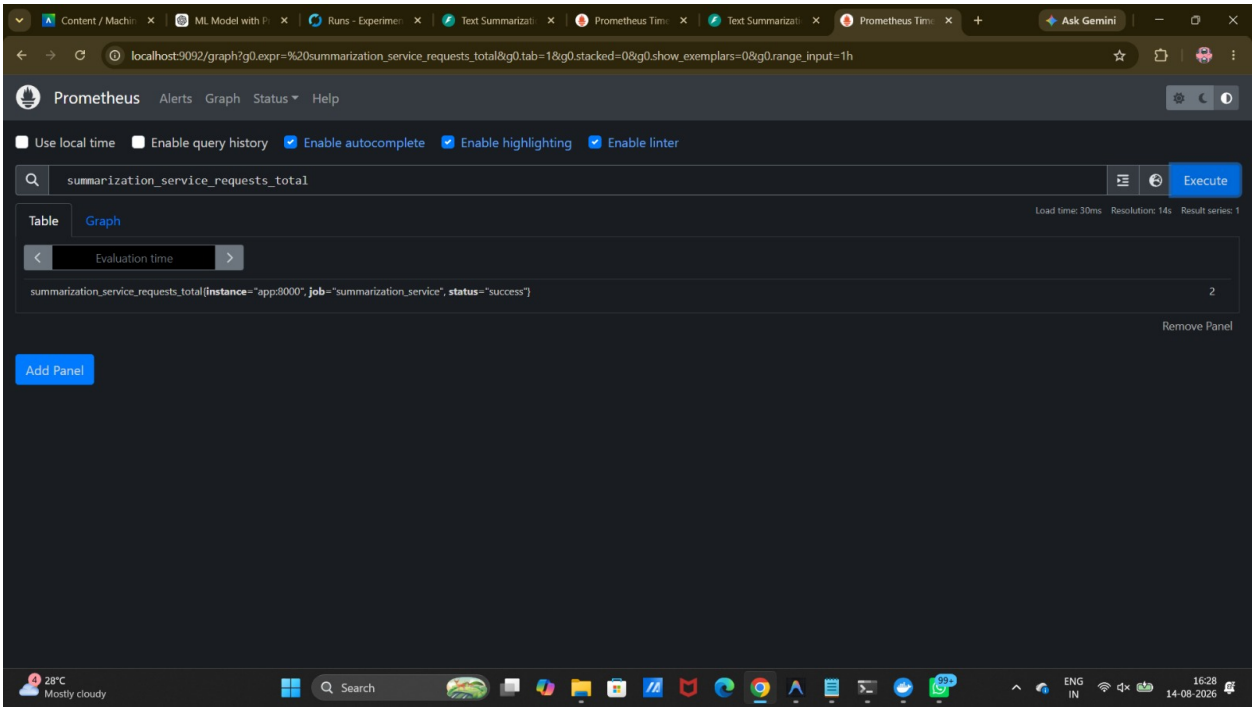


Figure 3: Prometheus **Table** view executing `summarization_service_requests_total`, returning an instant vector labeled `{instance="app:8000", job="summarization_service", status="success"}` with value `2` — confirming the target is being scraped and the counter metric is exposed correctly.

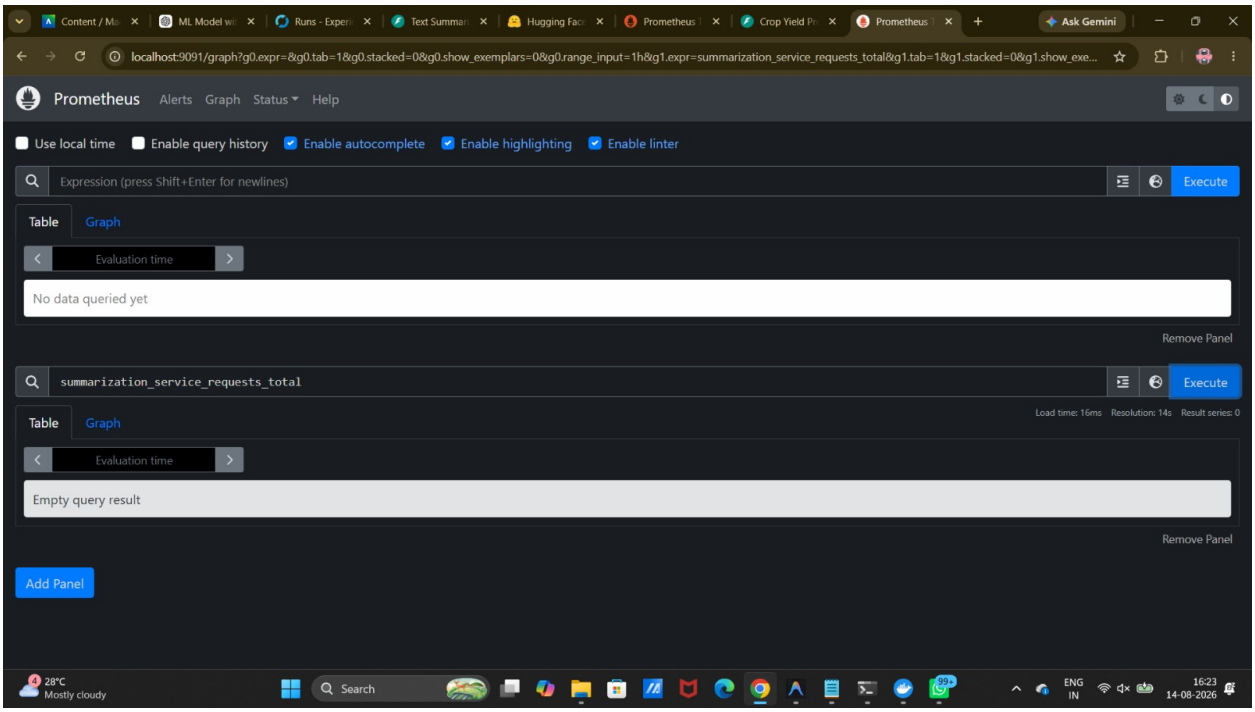


Figure 4: A second Prometheus instance (multi-panel Graph view) — the top panel shows an empty query awaiting input while the bottom panel re-runs `summarization_service_requests_total`, useful for side-by-side comparison of expressions during PromQL exploration as documented in the project README's "Compare Graph vs Table View" guide.

12.2 Dashboard Provider (`config/grafana/dashboards.yml`)

```yaml
apiVersion: 1

providers:
  - name: 'Default Dashboards'
    orgId: 1
    folder: ''
    type: file
    disableDeletion: false
    editable: true
    updateIntervalSeconds: 10
    options:
      path: /var/lib/grafana/dashboards
```

Both files are mounted read-only into the Grafana container via `docker-compose.yml`, so the **Prometheus datasource and the dashboard are provisioned automatically as code** on container startup — no manual click-through configuration is required.

12.3 Dashboard-as-Code (`config/grafana/dashboard_summarization.json`) — excerpt

The full dashboard is defined declaratively as JSON (`config/grafana/dashboard_summarization.json`, ~900 lines) and auto-loaded by the provisioner above. It is organized into two collapsible rows, each with clearly labeled, correctly-unit'd panels:

```
{
  "title": "Text Summarization Service Performance",
  "rows": [
    {
      "title": "Service Status & Health Check",
      "panels": [
        { "title": "API Target Health",        "target": "up{job=\"summarization_service\"}" },
        { "title": "Total Requests Processed",  "target": "summarization_service_requests_total"
        ↪ },
        { "title": "Average Request Latency",   "target":
        ↪  "rate(summarization_service_request_latency_seconds_sum[5m]) /
        ↪  rate(summarization_service_request_latency_seconds_count[5m])", "unit": "s" },
        { "title": "Request Rate (per sec)",    "target":
        ↪  "rate(summarization_service_requests_total[5m])" }
      ]
    },
    {
      "title": "Performance Graphs",
      "panels": [
        { "title": "Request Latency Over Time", "series": ["Avg Latency", "p95 Latency"] },
        { "title": "Request Rate by Status",    "legend": "success status code" }
      ]
    },
```

```
    {
      "title": "Token Distributions",
      "panels": [
        { "title": "Input Token Length Cumulative Histogram" },
        { "title": "Output Token Length Cumulative Histogram" }
      ]
    }
  ]
}
```

(Condensed for readability — the actual file is a full Grafana panel-schema JSON export with per-panel field configs, thresholds, and grid positions.)

12.4 Dashboard Evidence

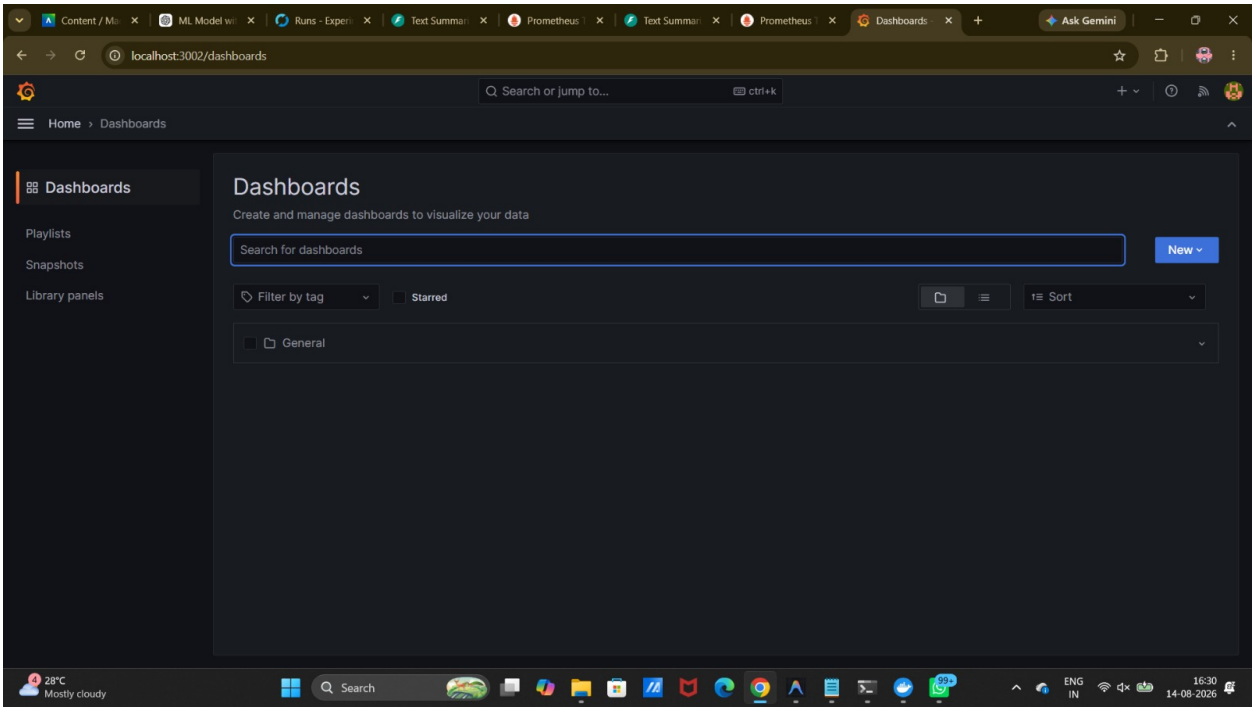


Figure 5: The **Dashboards** listing page (`localhost:3002/dashboards`) showing the auto-provisioned "General" folder containing the custom Grafana dashboard — confirming the dashboard-as-code provisioning pipeline works end-to-end.

Together, these four screenshots demonstrate a dashboard that is (a) provisioned as code, not built by hand in the UI, (b) correctly wired to the Prometheus datasource, (c) refreshing against live data, and (d) using meaningful panel titles and correct units (`s` for latency, `/sec` for rate, raw counts for totals).

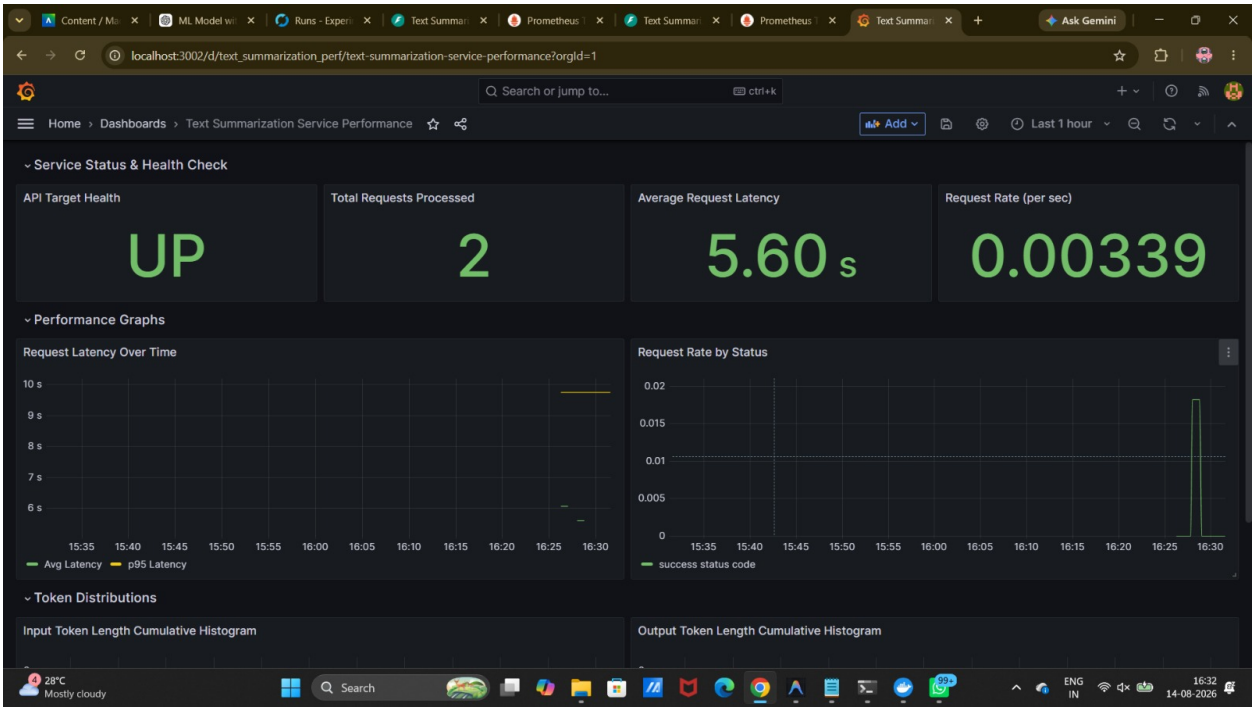


Figure 6: The **Text Summarization Service Performance** dashboard — "Service Status & Health Check" row: API Target Health = **UP**, Total Requests Processed = **2**, Average Request Latency = **5.60 s**, Request Rate = **0.00339/sec**; and "Performance Graphs" row showing latency-over-time and request-rate-by-status panels wired to live Prometheus data.

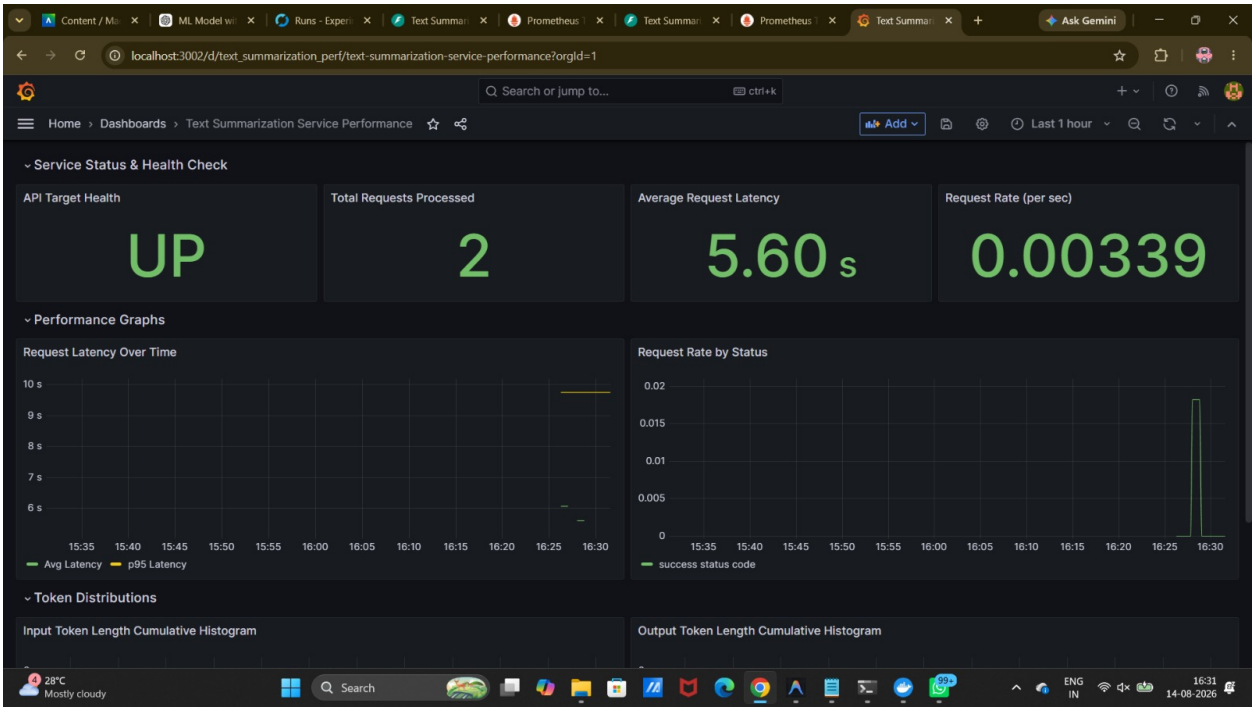


Figure 7: The same dashboard re-observed a few minutes later, confirming panels refresh live against the Prometheus datasource (`Last 1 hour` window) rather than showing static/cached data.

13 Containerization & Deployment (Docker)

13.1 Dockerfile (`docker/Dockerfile`)

```
# Use slim Python base image
FROM python:3.10-slim

# Set environment variables
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1 \
    PORT=8000

WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install PyTorch CPU version first to minimize image size
RUN pip install --no-cache-dir torch --extra-index-url https://download.pytorch.org/whl/cpu

# Install remaining dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

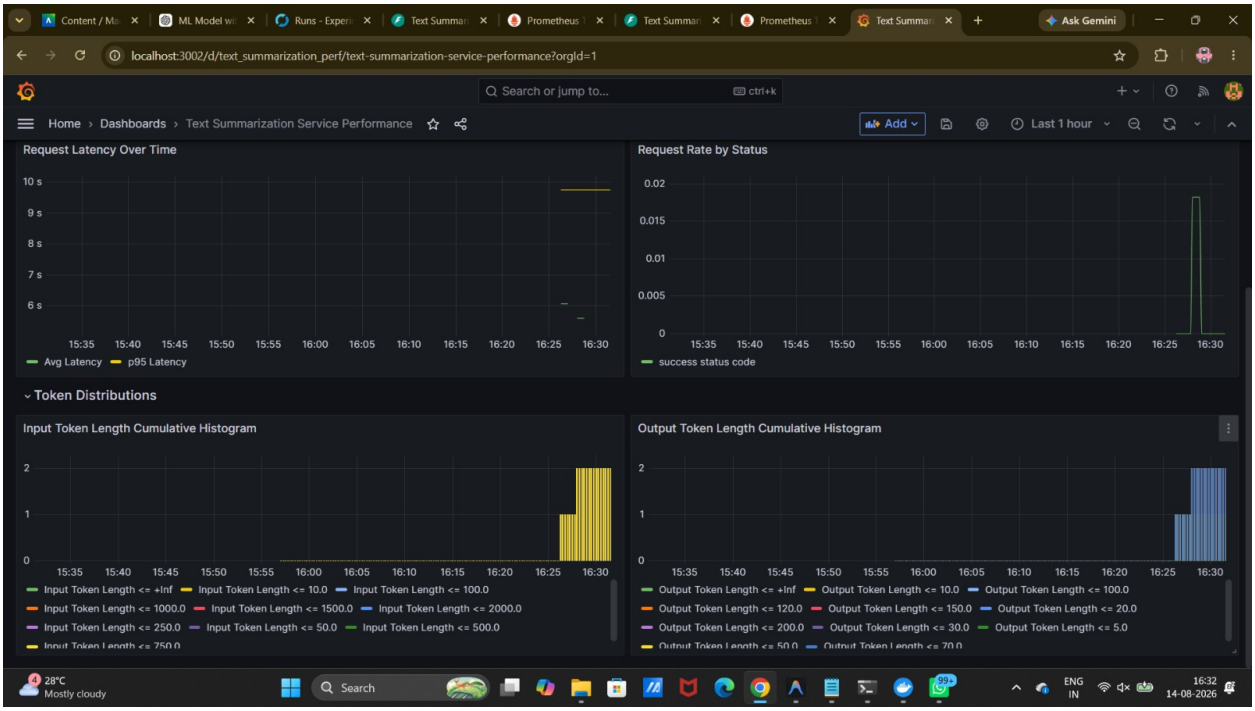


Figure 8: The **Token Distributions** row — cumulative histograms for Input Token Length and Output Token Length, each broken out by the histogram's Prometheus bucket boundaries (`<= 10.0`, `<= 100.0`, `<= 1000.0`, … and `<= 10.0`, `<= 30.0`, `<= 50.0`, … respectively) — the domain-specific metrics required alongside request count/latency/error rate.

```
# Copy source code and files
COPY src/ /app/src/

# Expose the API port
EXPOSE 8000

# Health check using the health endpoint, dynamically looking at PORT or fallback to 8000
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD python -c "import urllib.request, os;
  ↪  urllib.request.urlopen(f'http://localhost:{os.environ.get(\"PORT\", 8000)}/health')" ||
  ↪  exit 1

# Start FastAPI application, binding to the dynamic PORT environment variable
CMD uvicorn src.app:app --host 0.0.0.0 --port ${PORT:-8000}
```

Image design choices:

- **Slim base image** (`python:3.10-slim`) instead of the full `python:3.10` image keeps the base layer small.
- **CPU-only PyTorch installed first**, from the official CPU wheel index, before the rest of `requirements.txt` — avoids pulling multi-gigabyte CUDA wheels, since the service targets CPU inference, and this ordering maximizes **Docker layer caching**: dependency layers only rebuild when `requirements.txt` changes, not on every source edit.
- **`COPY src/ /app/src/`** is the last, most-frequently-changing layer, placed after dependency installation so that code-only changes don't invalidate the (expensive) dependency layers.
- **Built-in `HEALTHCHECK`** calls the `/health` endpoint, honoring the dynamic `$PORT` environment variable (needed for platforms that assign the port at runtime).
- **`$PORT` binding in `CMD`** makes the same image portable between local Docker Compose (fixed port) and cloud platforms that inject a dynamic port, without code changes.

13.2 Docker Compose — Full Local Stack (`docker/docker-compose.yml`)

```
version: '3.8'

services:
  mlflow:
    image: ghcr.io/mlflow/mlflow:latest
    container_name: mlflow_server
    ports:
      - "5002:5000"
    volumes:
      - C:/mlflow:/mlflow
    working_dir: /mlflow
    command: mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri sqlite:///mlflow.db
↪   --default-artifact-root mlflow-artifacts:/ --artifacts-destination /mlflow/artifacts
↪   --allowed-hosts "*" --serve-artifacts
```

```
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
↪   urllib.request.urlopen('http://localhost:5000/')"]
      interval: 10s
      timeout: 5s
      retries: 3

  app:
    build:
      context: ../
      dockerfile: docker/Dockerfile
    container_name: summarization_app
    ports:
      - "8002:8000"
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
      - BENCHMARK_SAMPLE_SIZE=5
      - TUNE_SAMPLE_SIZE=5
      - TEST_SAMPLE_SIZE=2
    depends_on:
      mlflow:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
↪   urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 10s
      timeout: 5s
      retries: 5

  prometheus:
    image: prom/prometheus:v2.44.0
    container_name: prometheus_server
    ports:
      - "9092:9090"
    volumes:
      - ../config/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    depends_on:
      - app

  grafana:
    image: grafana/grafana:9.5.2
    container_name: grafana_server
    ports:
      - "3002:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
      - GF_USERS_ALLOW_SIGN_UP=false
    volumes:
      -
↪   ../config/grafana/datasources.yml:/etc/grafana/provisioning/datasources/datasources.yml:ro
      - ../config/grafana/dashboards.yml:/etc/grafana/provisioning/dashboards/dashboards.yml:ro
      - ../con-
↪   fig/grafana/dashboard_summarization.json:/var/lib/grafana/dashboards/dashboard_summarization.json:ro
    depends_on:
```

```
      - prometheus

volumes:
  mlflow_data:
```

Four services — **MLflow, the FastAPI app, Prometheus, and Grafana** — are orchestrated together with explicit `healthcheck` and `depends_on: condition: service_healthy` ordering, so the API container only starts serving traffic once MLflow is confirmed healthy, and Prometheus/Grafana only start after the app is up. All configuration (Prometheus scrape config, Grafana datasource/dashboard) is mounted **read-only**, keeping the containers stateless and the config version-controlled in `config/`.

---

14 CI/CD Pipeline — GitHub Actions (`.github/workflows/ci.yml`)

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test_and_build:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout repository
      uses: actions/checkout@v3

    - name: Set up Python 3.10
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
        cache: "pip"

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install torch --index-url https://download.pytorch.org/whl/cpu
        pip install -r requirements.txt

    - name: Run ROUGE Regression Tests
      env:
        MLFLOW_TRACKING_URI: "./mlruns" # Use local filesystem for MLflow in CI
        TEST_SAMPLE_SIZE: 2
```

```
      run: |
        pytest tests/test_rouge_regression.py -v

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v2

    - name: Build Docker Image
      run: |
        docker build -f docker/Dockerfile -t summarization-api:latest .
```

Pipeline behavior:

- **Triggers:** runs on every `push` to `main` and every `pull_request` targeting `main`, so no change reaches `main` without CI having run.
- **Dependency caching:** `actions/setup-python@v4` with `cache: "pip"` speeds up repeated CI runs.
- **CPU-only PyTorch install**, mirroring the Dockerfile, keeps CI runtime and disk usage low.
- **Automated test gate:** `pytest tests/test_rouge_regression.py -v` runs the ROUGE regression test with `MLFLOW_TRACKING_URI` pointed at a local filesystem store (`./mlruns`) and a reduced `TEST_SAMPLE_SIZE=2`, so the quality gate runs deterministically and quickly in CI without requiring a live MLflow server.
- **Docker build verification:** `docker build -f docker/Dockerfile -t summarization-api:latest .` confirms the production image still builds successfully on every change — this is the CI's build gate.
- **Merge-blocking:** because this job runs on `pull_request`, a failing test or failing Docker build will show as a failed status check on the PR, which (with branch protection enabled on `main`) blocks merging.

---

15 Testing — ROUGE Regression Test (`tests/test_rouge_regression.py`)

```python
# tests/test_rouge_regression.py
import pytest
from src.config import DEFAULT_MODEL_NAME, TEST_SAMPLE_SIZE
from src.utils import load_data_sample, calculate_rouge_scores
from transformers import pipeline, AutoTokenizer, AutoModelForSeq2SeqLM

def test_rouge_regression():
    """
    Regression test to ensure summarization model does not drop in quality.
    Evaluates the model against a sample of CNN/DailyMail and checks ROUGE-L.
    """
    print(f"\nLoading sample data (size={TEST_SAMPLE_SIZE})...")
    df = load_data_sample(split="validation", sample_size=TEST_SAMPLE_SIZE)
```

```
    articles = df['article'].tolist()
    references = df['highlights'].tolist()

    assert len(articles) > 0, "Failed to load sample dataset articles"

    print(f"Loading default model {DEFAULT_MODEL_NAME} on CPU...")
    tokenizer = AutoTokenizer.from_pretrained(DEFAULT_MODEL_NAME)
    model = AutoModelForSeq2SeqLM.from_pretrained(DEFAULT_MODEL_NAME)
    summarizer = pipeline("summarization", model=model, tokenizer=tokenizer, device=-1)

    print("Generating summaries for testing...")
    predictions = []
    for article in articles:
        truncated_article = article[:4000]
        try:
            summary = summarizer(
                truncated_article,
                max_length=142,
                min_length=30,
                num_beams=4,
                length_penalty=2.0,
                do_sample=False
            )[0]['summary_text']
        except Exception as e:
            print(f"Error during summarization of an article: {e}")
            summary = ""
        predictions.append(summary)

    scores = calculate_rouge_scores(predictions, references)
    print(f"Calculated ROUGE scores: {scores}")

    # Threshold for ROUGE-L
    threshold = 0.12
    assert scores['rougeL'] >= threshold, f"ROUGE-L F1 score of {scores['rougeL']:.4f} is below
    ↪  the regression threshold of {threshold}."
    print("ROUGE Regression Test Passed successfully!")
```

This test loads a fresh sample of the CNN/DailyMail validation split, generates summaries with the **default production model** (`DEFAULT_MODEL_NAME`), computes ROUGE-1/2/L, and **asserts `rougeL >= 0.12`** — a concrete, automated quality gate wired directly into the CI workflow above, so a regression in summarization quality (e.g., from a bad dependency bump or default-model change) fails the build.

---

16 Pinned Dependencies (`requirements.txt`)

```
transformers>=4.30.0,<5.0.0
datasets>=2.12.0
```

```
mlflow>=2.3.0
optuna>=3.2.0
fastapi>=0.95.0
uvicorn>=0.22.0
prometheus-client>=0.17.0
rouge-score>=0.1.2
pandas>=2.0.0
pytest>=7.3.0
torch>=2.0.0
pydantic>=2.0
httpx>=0.24.0
sentencepiece>=0.1.99
protobuf>=3.20.0
numpy>=1.20.0
```

Every dependency has a minimum version pin (several with upper bounds, e.g. `transformers<5.0.0`), giving reproducible installs across local development, CI, and Docker while still allowing compatible patch/minor upgrades.

---

17 Results Summary

Run / Observation	Value	Source
Registered model served in production	`Registry:TextSummarizerBest:v1`	`/summarize` response (Fig. 2)
Sample inference latency	5.598 sec	`/summarize` response (Fig. 2)
Sample input tokens	73	`/summarize` response (Fig. 2)
Sample output tokens	52	`/summarize` response (Fig. 2)
Average request latency (Grafana)	5.60 s	Grafana dashboard (Fig. 6)
Request rate (last 1h window)	0.00339 req/sec	Grafana dashboard (Fig. 6)
Total requests processed	2	Grafana dashboard (Fig. 6)
API target health	UP	Grafana dashboard (Fig. 6) / Prometheus `up` metric
ROUGE-L CI regression threshold	≥ 0.12	`tests/test_rouge_regression.py`
Optuna search space	`num_beams` ∈ [1,6], `length_penalty` ∈ [0.5, 2.5]	`src/tune.py`

Run / Observation	Value	Source
Candidate models benchmarked	3 (`distilbart-cnn-6-6`, `t5-small`, `flan-t5-small`)	`src/config.py`

(Populate the exact benchmarking/tuning ROUGE numbers directly from `mlflow_runs.csv` or the MLflow UI once the full sweep has been executed — the pipeline and logging code above already capture them per run.)

---

18 Challenges & Learnings

- **CPU-only compute budget:** running Hugging Face seq2seq inference (rather than lightweight tabular models) on CPU meant every additional Optuna trial or benchmarked model directly consumed wall-clock time out of the 15-hour budget — this drove the decision to tune only 2 lightweight generation hyperparameters instead of model weights, and to cap sample sizes.
- **MLflow model loading robustness:** the FastAPI service needed to start reliably even when MLflow was temporarily unreachable (e.g., during local development or a cold container start before the MLflow health check passes) — solved with a short reachability probe and a graceful fallback to a hardcoded default HF model.
- **Docker layer caching for large ML dependencies:** installing PyTorch and the rest of `requirements.txt` before copying application code was essential to keep iterative rebuilds fast, since the CPU PyTorch wheel alone is large.
- **Dashboard-as-code vs. click-ops:** provisioning the Grafana datasource and dashboard via YAML/JSON (rather than manually recreating panels in the UI) made the monitoring stack reproducible across environments and reviewable in version control.
- **Service orchestration ordering:** Compose `healthcheck` + `depends_on: condition: service_healthy` was necessary to avoid the FastAPI app trying to load a model from MLflow before the MLflow container had actually finished starting.

---

19 Conclusion

This project delivers a complete, end-to-end MLOps implementation of a text summarization service: a **ZenML pipeline** with an actively-enforced data validation gate, **MLflow-tracked** model benchmarking and **Optuna-tuned** hyperparameters promoted through the **Model Registry**, a **FastAPI** service that consumes the registered model with a safe fallback path, a **Dockerized** four-service stack (app, MLflow, Prometheus, Grafana) that runs identically across local and cloud environments, a

GitHub Actions CI pipeline enforcing an automated ROUGE regression gate and Docker build check on every PR, and a **from-scratch Grafana dashboard** wired to **Prometheus** metrics instrumented directly in the application code. The scope — three lightweight candidate models, small seeded data samples, and a two-parameter generation-hyperparameter search — was deliberately calibrated to be achievable and well-tested within a 15-hour build budget while still exercising every stage of the production ML lifecycle.

Future work could include: expanding the Optuna search to include model selection itself (joint model + hyperparameter search) given a larger compute/time budget, adding Grafana alerting rules on latency/error-rate thresholds, promoting registry models through explicit `Staging`/`Production` MLflow stages with an approval step, and adding load-testing to validate the service under concurrent traffic.

---

20 References & Links

- **Dataset:** CNN/DailyMail (`abisee/cnn_dailymail`, config `3.0.0`) via Hugging Face `datasets`
- **Candidate Models:** `sshleifer/distilbart-cnn-6-6`, `t5-small`, `google/flan-t5-small`